

Министерство образования и науки Российской Федерации
Федеральное Государственное Автономное Образовательное
Учреждение Высшего Образования "Национальный
Исследовательский Университет ИТМО"

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

По дисциплине "Тестирование программного обеспечения"

Вариант: 34209

Выполнил студент группы Р3319

Алексеев Павел Сергеевич

Проверил преподаватель практики

Тюрин Иван Николаевич

г. Санкт-Петербург, 2026 г.

Условие

Лабораторная работа #1

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Введите вариант:

1. Функция $\text{tg}(x)$
2. Программный модуль для поиска кратчайшего пути в ориентированном графе по алгоритму Дейкстры (<http://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>)
3. Описание предметной области:

Отсюда такая фраза из жаргона автостопщиков, как: "Послушай, ты зючишь этого хипеля Форда Префекта? Это тот еще фрокт, он всегда при полотенце". (Зючить: знать, быть слышанным, встречать, иметь сексуальные отношения; хипель: солидно упакованный парень; фрокт: реально солидно упакованный парень.)

Задание 1

Функция - $\tan(x)$

1. Область определения функции: $x \in R \setminus \{\frac{\pi}{2} + \pi * k, k \in Z\}$

2. Область значений функции: $f(x) \in (-\infty; +\infty)$

3. Вертикальные асимптоты:

- $\lim_{x \rightarrow \frac{\pi}{2} - 0} f(x) = +\infty$

- $\lim_{x \rightarrow \frac{\pi}{2} + 0} f(x) = -\infty$

Тестирование проверяет корректность реализации функции тангенса через ряд Тейлора. В качестве эталонных значений взяты результаты выполнения вычислений на калькуляторе, встроенном в Windows 10.

Тест 1 (famousValuesTest):

вычисляет значения тангенса для известных значений(таких как 0, $\frac{\pi}{4}$, $\frac{\pi}{3}$, $\frac{\pi}{6}$)

Тест 2 (infinityTest):

проверка передачи бесконечностей в качестве аргумента функции(Результат Nan).

Тест 3 (nanTest):

проверка передачи Nan в качестве аргумента функции(Результат Nan).

Тест 4 (bigValuesTest):

Проверка больших значений(1000, 1000000) и их корректную обработку.

Задание 2

Тестирование проверяет корректность реализации алгоритма Дейкстры на ориентированном графе. Алгоритм возвращает трассу сделанных действий а также финальную таблицу с расстояниями до всех возможных вершин из заданной. ОДЗ - веса ребер графа неотрицательны.

Тест 1 (emptyGraphTest):

Проверка передачи в алгоритм пустого графа. Ожидаемое поведение - `IllegalArgumentException`.

Тест 2 (vGreaterThanGraphSizeTest):

Проверка передачи номера вершины, не существующей в графе. Ожидаемое поведение - `IllegalArgumentException`.

Тест 3 (linearGraphTest):

Проверка поведения алгоритма на простом дереве(у одной вершины один потомок).

Тест 4 (graphFromSite0Test):

Проверка поведения алгоритма на графе с 8 вершинами(старт в 0 вершине).

Тест 5 (linearGraphTest):

Проверка поведения алгоритма на графе с 8 вершинами(старт в 5 вершине).

Задание 3

Тестирование проверяет логику перехода объектов из одного состояния.

Состояния и условия перехода основаны на текстовом описании предметной области.

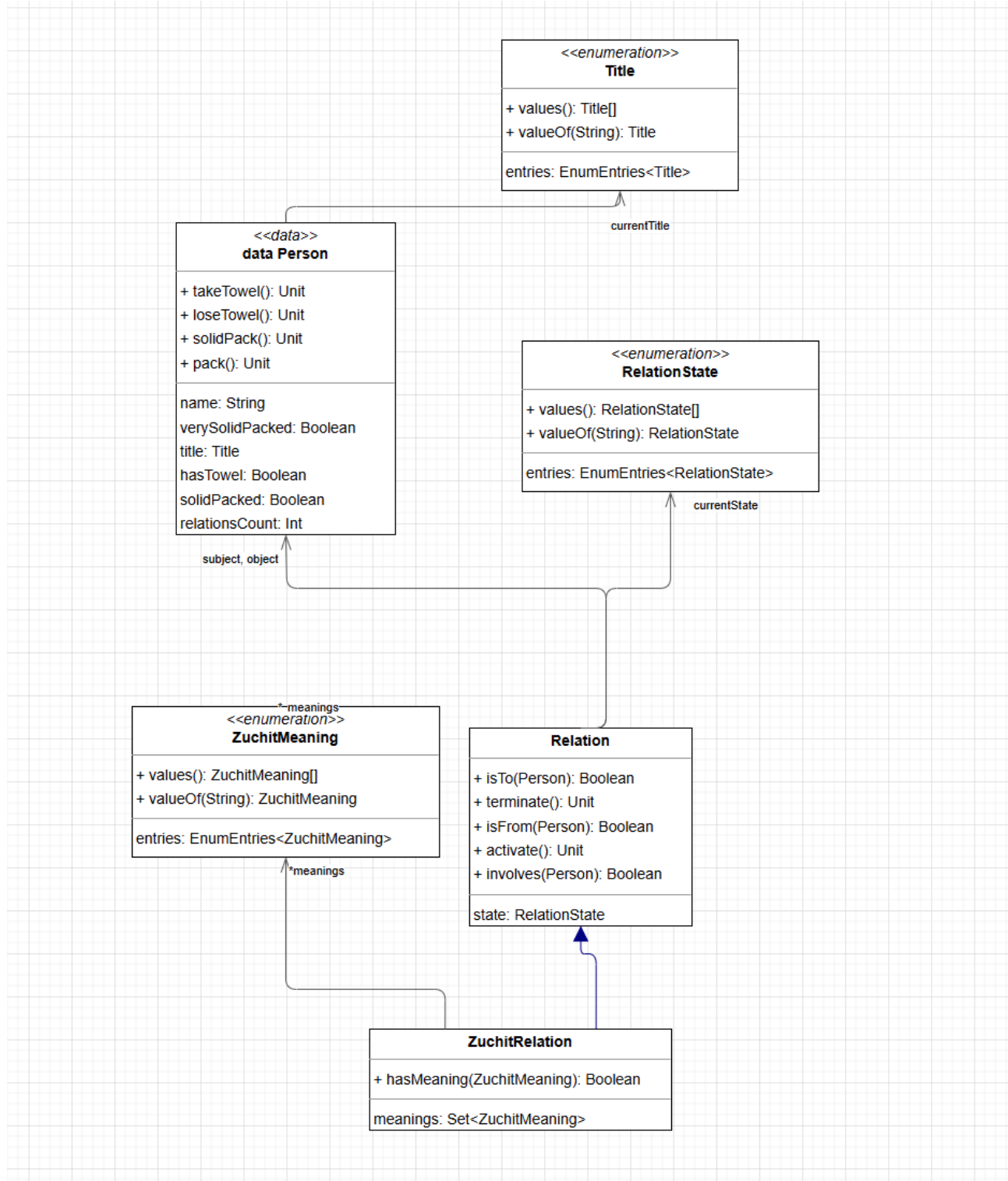


Figure 1: UML-диаграмма классов для задания 3

Тестирование предметной области

Набор тестов проверяет корректность построения отношений между персонажами и изменения их состояний в зависимости от исследуемого сценария.

Тесты класса PersonTest:

Проверка всевозможных сценариев с участием человека(Person) - создание с некорректными параметрами, проверка дефолтных значений, процессы перехода из состояния Default в состояния Hipel и Frockt, тест наличия и отсутствия полотенца.

Тесты класса RelationTest:

Проверка всевозможных сценариев с образованием зависимости между двумя людьми(Relation, ZuchitRelation) - проверка жизненного цикла Relation(состояния created, active, terminated), проверка изменения состояния участников зависимости и способов определения принадлежности участника к зависимости. Для ZuchitRelation - проверка обращения с множеством значения зависимости, проверка образования этой зависимости с самим собой, проверка сохранения жизненного цикла.

Вывод

В ходе выполнения лабораторной работы я познакомился с базовыми возможностями JUnit 5 на примере разработки модульного тестового покрытия для программ трёх типов: математическая функция, алгоритм и сценарий на базе описанной предметной области. Наиболее интересным мне показалось многообразие assert'ов для проверки и общая структура(прозрачная и гибкая) написания тестов.